Code:

```
#include <stdio.h>
int main(){
    int val  = 53;
    int *ptr = &val;
    int a[5] = {10, 20, 30, 40, 50};
    int idx = 2;

    printf("Immediate Addressing mode: %d\n", 100);
    printf("Direct Addressing mode: %d\n", val);
    printf("Indirect addressing: %d\n", *ptr);
    int reg = val;
    printf("Register addressing: %d\n", reg);
    printf("Indexed addressing: %d\n", a[idx]);
    printf("Base + Offset Addressing: %d\n", *(a + 3));
    return 0;
}
```

Output:

```
PS C:\Users\LAB 102 PC 25\Desktop\241210059logisim\COAvscode> gdb ./addrem
GNU gdb (GDB) 7.6.1
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "mingw32".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from C:\Users\LAB 102 PC
25\Desktop\241210059logisim\COAvscode\addrem.exe...done.
(gdb) break main
Breakpoint 1 at 0x40146e: file addrem.c, line 4.
(gdb) run
Starting program: C:\Users\LAB 102 PC
25\Desktop\241210059logisim\COAvscode/./addrem.exe
[New Thread 8280.0x1450]
[New Thread 8280.0xca8]
```

```
Breakpoint 1, main () at addrem.c:4
warning: Source file is more recent than executable.
4           int val  = 53;
(gdb) disassemble /m main
Dump of assembler code for function main:
3       int main(){
   0x00401460 <+0>:     push   %ebp
   0x00401461 <+1>:     mov    %esp,%ebp
   0x00401463 <+3>:     and    $0xfffffff0,%esp
   0x00401466 <+6>:     sub    $0x40,%esp
   0x00401469 <+9>:     call   0x401a70 <__main>

4           int val  = 53;
=> 0x0040146e <+14>:    movl   $0x35,0x30(%esp)

5           int *ptr = &val;
   0x00401476 <+22>:    lea    0x30(%esp),%eax
   0x0040147a <+26>:    mov    %eax,0x3c(%esp)

6           int a[5] = {10, 20, 30, 40, 50};
   0x0040147e <+30>:    movl   $0xa,0x1c(%esp)
   0x00401486 <+38>:    movl   $0x14,0x20(%esp)
   0x0040148e <+46>:    movl   $0x1e,0x24(%esp)
   0x00401496 <+54>:    movl   $0x28,0x28(%esp)
   0x0040149e <+62>:    movl   $0x32,0x2c(%esp)

7           int idx = 2;
   0x004014a6 <+70>:    movl   $0x2,0x38(%esp)

8
9           printf("Immediate Addressing mode: %d\n", 100);
   0x004014ae <+78>:    movl   $0x64,0x4(%esp)
   0x004014b6 <+86>:    movl   $0x405064,(%esp)
   0x004014bd <+93>:    call   0x403b10 <printf>

10          printf("Direct Addressing mode: %d\n", val);
   0x004014c2 <+98>:    mov    0x30(%esp),%eax
   0x004014c6 <+102>:   mov    %eax,0x4(%esp)
   0x004014ca <+106>:   movl   $0x405083,(%esp)
   0x004014d1 <+113>:   call   0x403b10 <printf>

11          printf("Indirect addressing: %d\n", *ptr);
   0x004014d6 <+118>:   mov    0x3c(%esp),%eax
```

```
   0x004014da <+122>:   mov    (%eax),%eax
   0x004014dc <+124>:   mov    %eax,0x4(%esp)
   0x004014e0 <+128>:   movl   $0x40509f,(%esp)
   0x004014e7 <+135>:   call   0x403b10 <printf>

12         int reg = val;
   0x004014ec <+140>:   mov    0x30(%esp),%eax
   0x004014f0 <+144>:   mov    %eax,0x34(%esp)

13         printf("Register addressing: %d\n", reg);
   0x004014f4 <+148>:   mov    0x34(%esp),%eax
   0x004014f8 <+152>:   mov    %eax,0x4(%esp)
   0x004014fc <+156>:   movl   $0x4050b8,(%esp)
   0x00401503 <+163>:   call   0x403b10 <printf>

14         printf("Indexed addressing: %d\n", a[idx]);
   0x00401508 <+168>:   mov    0x38(%esp),%eax
   0x0040150c <+172>:   mov    0x1c(%esp,%eax,4),%eax
   0x00401510 <+176>:   mov    %eax,0x4(%esp)
   0x00401514 <+180>:   movl   $0x4050d1,(%esp)
   0x0040151b <+187>:   call   0x403b10 <printf>

15         printf("Base + Offset Addressing: %d\n", *(a + 3));
   0x00401520 <+192>:   mov    0x28(%esp),%eax
   0x00401524 <+196>:   mov    %eax,0x4(%esp)
   0x00401528 <+200>:   movl   $0x4050e9,(%esp)
   0x0040152f <+207>:   call   0x403b10 <printf>

16         return 0;
   0x00401534 <+212>:   mov    $0x0,%eax

17     }   0x00401539 <+217>:  leave
   0x0040153a <+218>:   ret

End of assembler dump.
(gdb)
```